

FRESHER TO FULL-STACK DEVELOPER GUIDE

FASTAPI ZERO TO HERO

Project-Based Master Handbook: Building the Oranza Digital Catalog Backend

Ek complete step-by-step practical guide jisme ek fresher beginner level se Python, FastAPI, SQLAlchemy ORM, SQLite, JWT Authentication, aur Media Uploads ko real-world project ke zariye seekh sakta hai.

Project: Ali-Eco-Cat / Oranza Living & Lifestyle

Tech Stack: Python 3.13 + FastAPI + SQLAlchemy 2.0 + SQLite + Pydantic v2 + React 18

Total APIs Implemented: 25 Production-Ready REST Endpoints

Table of Contents

1. **Chapter 1:** Web Architecture & FastAPI Fundamentals (API Flow)
2. **Chapter 2:** Project Directory Structure & Core Files Role
3. **Chapter 3:** Database Connection & Session Management (database.py)
4. **Chapter 4:** Authentication, Hashing & JWT Security (auth.py)
5. **Chapter 5:** Categories Module (Taxonomy & Organization)
6. **Chapter 6:** Products Engine: Search, Filtering & View Tracking
7. **Chapter 7:** WhatsApp Consultation & Inquiry Click Tracker
8. **Chapter 8:** Media File Uploads (Device Image Picker to Live URLs)
9. **Chapter 9:** Dashboard Analytics & SQL Aggregation Metrics
10. **Chapter 10:** Customer CRM Leads & Contact Inquiries
11. **Chapter 11:** Store & Company Global Configuration (Singleton Pattern)
12. **Chapter 12:** Excel & CSV Data Export Engine (StreamingResponse)
13. **Chapter 13:** Top Interview Questions & Common Beginner Bugs Fixes

Chapter 1: Web Architecture & FastAPI Fundamentals

Jab bhi hum koi modern web application (jaise Amazon, Flipkart, ya hamara Oranza Catalog) banate hain, to uske **do main hisse** hote hain:

- **Frontend (Client):** Jo user ko browser me screen par dikhta hai (React, HTML, CSS). Iska kaam sundar buttons, forms aur cards render karna hai.
- **Backend (Server):** Jo server par chalta hai (Python + FastAPI). Iska kaam business logic execute karna, security check karna, aur database me data save/fetch karna hai.

API kya hoti hai? (Beginner Analogy)

API (Application Programming Interface) ek **Restaurant ke Waiter** ki tarah hoti hai:

Client (Customer): Menu dekh kar waiter ko order deta hai → **HTTP Request**

API (Waiter): Order ko kitchen (Database/Backend) tak le jata hai.

Backend (Kitchen): Khana (Data) prepare karta hai.

API (Waiter): Tayyar khana customer ki table par wapas le aata hai → **HTTP Response (JSON)** .

Complete Request-Response Lifecycle Diagram



Chapter 2: Project Directory Structure & Core Files

Hamare backend ka code clean modular architecture par structured hai:

File Name	Primary Responsibility	Beginner Explanation
database.py	DB Connection & Engine	Database file (catalog.db) ka connection banata hai aur session provide karta hai.
models.py	SQLAlchemy ORM Tables	Database ki actual tables (Users, Products, Categories, Settings) define karta hai.
schemas.py	Pydantic Data Schemas	API me aane wale aur jane wale JSON data ka data-type aur validation check karta hai.
crud.py	Database Operations	Actual SQL queries (Create, Read, Update, Delete) perform karne wale Python functions.
auth.py	Security & JWT Tokens	Bcrypt password hashing aur JWT Bearer token generation/verification handle karta hai.
main.py	FastAPI Endpoints Entrypoint	Saare URLs/routes ko register karta hai, CORS handle karta hai aur server run karta hai.

models.py vs schemas.py ka Difference (Sabse Bada Beginner Confusion)

models.py (SQL Database Table): Ye database engine ko batata hai ki disk par hard drive me table kaise banegi (Columns: Integer, String, Primary Key, Foreign Key).

schemas.py (Pydantic Validator): Ye network layer par client se aane wale JSON ko check karta hai (Jaise: email valid format me hai ya nahi, password kam se kam 6 characters ka hai ya nahi).

Chapter 3: Database Connection & Session Management (database.py)

Database se connect karne ke liye hum **SQLAlchemy ORM** ka use karte hain. ORM ka full form hai *Object Relational Mapper*. Iska fayda ye hai ki hume manually complex SQL queries nahi likhni padti, balki Python classes aur objects ke zariye database handle hota hai!

database.py Code:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

# 1. Local SQLite file path
SQLALCHEMY_DATABASE_URL = "sqlite:///./catalog.db"

# 2. Database Engine create karna
engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False} # SQLite threading fix
)

# 3. SessionLocal: Har request ke liye DB session factory
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# 4. Declarative Base class jisse hamare saare models inherit karenge
Base = declarative_base()

# 5. Dependency Injection: get_db()
def get_db():
    db = SessionLocal()
    try:
        yield db # Route ko DB session hand-over karta hai
    finally:
        db.close() # Request poori hote hi session safely close ho jata hai
```

Key Learning (Dependency Injection - get_db): `yield` keyword ka use karke FastAPI request aane par darwaza (session) kholta hai aur request complete hone par `finally` me darwaza band kar deta hai. Isse connection leak nahi hota!

Chapter 4: Authentication, Hashing & JWT Security (auth.py)

Frontend Connection:

Connects to `LoginPage.jsx` aur `RegisterPage.jsx` .

Real Problem:

Database me users ka password plain text (e.g. `password123`) me save karna dangerous hai! Agar database leak ho gaya to saare accounts hack ho jayenge. Iske alawa, HTTP stateless hota hai, isliye har request par server ko user ki identity verify karni hoti hai.

Solution (Bcrypt + JWT):

1. **Bcrypt Hashing:** One-way cryptographic hash jo password ko random encrypted string bana deta hai jise decrypt nahi kiya ja sakta.
2. **JWT (JSON Web Token):** Login hone par user ko ek digital identity pass milta hai: `eyJhbGciOi...`



Code Example (Role Guards):

```
def require_seller_or_admin(current_user = Depends(get_current_user)):
    # Only Sellers or Admins can access
    if current_user.role not in ["seller", "admin"]:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Access forbidden: Requires Seller or Admin privileges"
        )
    return current_user
```

Chapter 5 & 6: Products & Categories Catalog Engine

Frontend Connection:

HomePage.jsx , CatalogPage.jsx , ProductDetailPage.jsx , AddProductPage.jsx .

Key APIs:

Endpoint	Method	Access	Description
/api/v1/categories	GET	Public	Saari categories aur unke live product count return karta hai.
/api/v1/products	GET	Public	Category filter, keyword search, aur pagination handle karta hai.
/api/v1/products/{id}	GET	Public	Product detail view aur views counter auto +1 karta hai.
/api/v1/products	POST	 Seller/Admin	Naya product create karta hai (with JSON specifications & video URL).

Advanced Search & Filtering Logic (crud.py):

```
def get_products(db: Session, category_id=None, search=None, is_featured=None, skip=0, limit=50):
    query = db.query(models.Product)

    # 1. Category Filter
    if category_id:
        query = query.filter(models.Product.category_id == category_id)

    # 2. Case-Insensitive Multi-field Search
    if search:
        search_fmt = f"%{search.strip()}%"
        query = query.filter(
            or_(
                models.Product.title.ilike(search_fmt),
                models.Product.short_description.ilike(search_fmt),
                models.Product.sku.ilike(search_fmt)
            )
        )

    # 3. Featured Products Filter
    if is_featured is not None:
        query = query.filter(models.Product.is_featured == is_featured)

    # 4. Pagination (skip & limit)
    return query.offset(skip).limit(limit).all()
```

Chapter 7 & 8: WhatsApp Analytics & Image Uploads

7. WhatsApp Enquiry Click Tracker (POST /api/v1/enquiries/click)

Digital catalog me customer cart checkout karne ki jagah seedha WhatsApp button dabata hai. Backend me har click par us product ka `enquiries` column `+1` ho jata hai. Isse showroom owner ko dashboard par pata chalta hai ki kis product me sabse zyada customer interest hai!

8. File & Image Upload System (POST /api/v1/uploads/image)

Frontend Section: `AddProductPage.jsx` me computer se product photos upload karna.

Logic & Implementation Steps:

1. **File Extension Check:** Sirf valid images (`.jpg` , `.png` , `.webp`) allow honi chahiye.
2. **Collision-Safe Unique Filename:** Agar do users `photo.jpg` upload karenge to pehli photo overwrite na ho, iske liye hum `uuid.uuid4().hex[:8]` attach karte hain: `img_a1b2c3d4_sofa.jpg` .
3. **Disk Streaming:** Binary data ko `static/uploads/images/` folder me save karte hain.
4. **Live HTTP URL Generation:** Client ko direct public link return hota hai:
`http://127.0.0.1:8000/static/uploads/images/img_a1b2c3d4_sofa.jpg` .

```
# main.py me static folder mount karna
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")

@app.post("/api/v1/uploads/image")
async def upload_product_image(request: Request, file: UploadFile = File(...)):
    ext = os.path.splitext(file.filename)[1].lower()
    if ext not in {".jpg", ".jpeg", ".png", ".webp"}:
        raise HTTPException(status_code=400, detail="Invalid image format")

    unique_name = f"img_{uuid.uuid4().hex[:8]}_{file.filename}"
    save_path = os.path.join(UPLOADS_IMAGES_DIR, unique_name)

    with open(save_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    base_url = str(request.base_url).rstrip("/")
    return {
        "status": "success",
        "url": f"{base_url}/static/uploads/images/{unique_name}"
    }
```

Chapter 9 & 10: Dashboard Analytics & Customer Leads

9. Dashboard Live Metrics (GET /api/v1/analytics/dashboard)

Frontend Section: `DashboardPage.jsx` ke top 4 statistics cards (Total Products, Categories, Views, Enquiries) aur Top 5 ranking tables.

Fast SQL Aggregation Queries: Hazaron records ko Python me loop chalane ke bajaye SQL database engine se direct calculate karwaya jata hai:

```
db.query(func.coalesce(func.sum(models.Product.views), 0)).scalar()
```

10. Customer Inquiries CRM Engine

Frontend Section: `ContactPage.jsx` (Inquiry Form).

- `POST /api/v1/contact` → Customer apna naam, phone, interest aur message submit karta hai.
- `GET /api/v1/contact/messages` → Protected: Seller dashboard me saari leads latest-first order me display hoti hain.
- `PUT /api/v1/contact/messages/{id}/read` → Seller lead ko "Contacted / Read" mark kar deta hai.

Chapter 11 & 12: Global Settings & Excel CSV Export

11. Store & Company Settings (GET & PUT /api/v1/settings)

Frontend Section: `FloatingWhatsApp.jsx`, `Footer.jsx`, `ContactPage.jsx`.

Pehle showroom ka WhatsApp number, address aur email JavaScript file me hardcoded tha. Ab database me **Singleton Row (ID=1)** banti hai:

- Visitor aayega to `GET /api/v1/settings` se latest phone number fetch hoga.
- Owner dashboard se `PUT /api/v1/settings` karke apna address ya phone number ek click me live update kar sakta hai bina coding badle!

12. Excel / CSV Data Export Engine (GET /api/v1/contact/export/csv)

Frontend Section: Dashboard me `Export Leads to Excel` button.

Architecture: StreamingResponse with BOM

```
import csv
import io
from fastapi.responses import StreamingResponse

@app.get("/api/v1/contact/export/csv")
def export_leads_csv(db: Session = Depends(get_db), current_user = Depends(auth.require_seller_or_admin)):
    leads = db.query(models.ContactMessage).order_by(models.ContactMessage.id.desc()).all()

    # 1. Virtual in-memory string file (No disk write)
    output = io.StringIO()
    output.write('\ufeff') # UTF-8 BOM: Microsoft Excel compatibility fix
    writer = csv.writer(output)

    # 2. Write CSV Header & Rows
    writer.writerow(["ID", "Date", "Customer Name", "Phone", "Category", "Message"])
    for l in leads:
        writer.writerow([l.id, l.created_at, l.name, l.phone, l.category, l.message])

    filename = "oranza_leads.csv"
    return StreamingResponse(
        iter([output.getvalue()]),
        media_type="text/csv; charset=utf-8",
        headers={"Content-Disposition": f"attachment; filename={filename}"})
    )
```

Chapter 13: Fresher Cheat Sheet & Common Errors

Issue / Error	Why it happens?	How to Fix?
422 Unprocessable Entity	Pydantic validation fail ho gaya (e.g. integer field me string bhej di, ya required field miss kar di).	Response body me <code>detail[0].loc</code> check karo aur required payload verify karo.
401 Unauthorized	Request ke header me <code>Authorization: Bearer <token></code> nahi bheja ya token expire ho gaya.	Pehle <code>/auth/login</code> karke fresh token lo aur headers me pass karo.
403 Forbidden	User logged in hai lekin uske paas required role (e.g. admin) nahi hai.	Admin user account se login karo (<code>admin@oranza.com</code>).
CORS Error in React	Browser ne security ke liye port 3001 se port 8000 ki request block kar di.	FastAPI me <code>CORSMiddleware</code> lagao with <code>allow_origins=["*"]</code> .
UnicodeEncodeError in Windows	Windows terminal cp1252 use karta hai aur special symbol (jaise ₹ ya emoji) print karne par crash hota hai.	Console prints me emojis avoid karein ya UTF-8 safe encode/decode karein.

Congratulations! 🚀

Aapne ek complete, industry-standard, secure FastAPI backend build aur master kar liya hai.

Created for Ali-Eco-Cat / Oranza Digital Catalog System